GenomeIQ — DNA Disease Classifier · Technical Documentation

Author: MD Naimur Rashid **Project type:** AI/ML capstone — DNA sequence classification with explainability, OOD detection, transformer integration, and RAG-powered chat.

Table of contents

1. What is this project
2. Tech stack at a glance
3. Repository layout
4. Data pipeline
5. ML models in detail
6. API endpoints
7. Frontend pages
8. Configuration & secrets
9. How to run everything
10. How to retrain / extend
11. Evaluation results
12. Glossary

---

1. What is this project

GenomeIQ takes a raw DNA sequence (a string of A, T, G, C characters) and predicts whether it carries genetic markers of one of four classes: **Cancer**, **Diabetes**, **Alzheimer's disease**, or **Normal** (housekeeping). On top of that prediction it provides:

- a **saliency map** showing which bases drove the verdict;
- an **out-of-distribution (OOD) check** so the user knows when to trust the prediction;
- a **nearest-neighbour search** (BLAST-lite) against the training corpus;
- a **mutation analyser** that compares two sequences and reports synonymous / missense / nonsense changes;
- a **batch FASTA upload** for multi-sequence classification;
- a **PDF report** of the analysis;
- a **RAG chat copilot** that answers questions using a curated knowledge base + an LLM.

The system has two prediction backbones — a classical **TF-IDF k-mer ensemble** and a **DNABERT-2 transformer** (frozen + fine-tuned). They can be averaged into an **Ensemble** mode.

---

2. Tech stack at a glance

Layer	Components
Backend	Python 3.10 · FastAPI · Uvicorn · Pydantic

Layer	Components
Classical ML	scikit-learn (RF, LinearSVC, LR, CalibratedClassifierCV) · TF-IDF
Transformer	PyTorch + Transformers · DNABERT-2 (`zhihan1996/DNABERT-2-117M`)
Bioinformatics	Biopython · custom k-mer / ORF / GC utilities
Vector store	sentence-transformers (`all-MiniLM-L6-v2`) + FAISS
LLM provider	Google Gemini 2.5 Flash (free) · Ollama (free local) · template demo (offline)
PDF report	ReportLab
Frontend	Tailwind (CDN) · Alpine.js · Chart.js · Plotly
Hardware	Tested on RTX 3050 4 GB (CUDA 12.4)

---

3. Repository layout

```
dna-disease-classifier/
├── data/
│   ├── raw/disease_sequences/disease_dna_dataset.csv      ← original 808
sequences
│   └── processed/                                          ← built by data
pipeline
│       ├── train.parquet, val.parquet                      ← augmented splits
│       ├── build_summary.json                              ← dataset audit
│       ├── cv_report_k6.json                               ← cross-validation
results
│       └── grid_search_report.json
├── models/
│   ├── disease_classifier.pkl, disease_vectorizer.pkl,     ← TF-IDF ensemble
│   │   disease_label_encoder.pkl
│   ├── dnabert2_base/                                      ← cached pretrained
weights
│   ├── dnabert2_finetuned/                                 ← fine-tuned variant
│   ├── dnabert2_embeddings.npz, dnabert2_head.pkl          ← frozen + LR head
│   ├── dnabert2_label_encoder.pkl
│   ├── similarity_index.npz, ood_stats.npz                 ← derived indices
│   ├── rag_index.faiss                                     ← knowledge-base FAISS
│   └── *_metrics.json, training_metrics.json               ← evaluation reports
├── src/
│   ├── core/
│   │   ├── config.py        constants, paths
│   │   ├── sequence.py      validation, GC, k-mers, ORFs, FASTA parsing
│   │   └── knowledge.py     disease/gene metadata
│   ├── data/
│   │   ├── audit.py         dataset diagnostics
│   │   ├── dedup.py         MinHash-Jaccard near-dup removal
│   │   ├── augment.py       sliding window + reverse-complement augmentation
│   │   └── build_dataset.py orchestrates clean → dedup → split → augment →
balance
```

```
│   ├── ml/
│   │   ├── classifier.py     TF-IDF ensemble loader + predictor
│   │   ├── explain.py        per-base saliency
│   │   ├── ood.py            OOD detection
│   │   ├── similarity.py     nearest-neighbour BLAST-lite
│   │   ├── mutation.py       reference vs variant comparison
│   │   ├── dnabert.py        DNABERT-2 frozen embeddings + LR head + fine-tuned
inference
│   │   └── dnabert_finetune.py  custom training loop tuned for 4 GB GPU
│   ├── rag/
│   │   ├── knowledge_base.py 29 curated facts (disease, gene, method)
│   │   ├── retriever.py      FAISS + sentence-transformers retrieval
│   │   ├── providers.py      demo / Gemini / Ollama LLM adapters
│   │   └── chat.py           orchestration
│   ├── api/
│   │   ├── app.py            FastAPI entry point + static UI mount
│   │   ├── schemas.py        Pydantic request/response models
│   │   ├── service.py        high-level orchestration
│   │   └── report.py         ReportLab PDF builder
│   ├── train.py              CLI to train TF-IDF ensemble
│   ├── eval.py               CLI for K-fold CV + grid search
│   ├── preprocess.py         simple processed-feature dump
│   └── predict.py            CLI predictor
├── frontend/
│   ├── index.html            single-page UI
│   └── static/{app.js, styles.css}
├── notebooks/
│   ├── 01_exploration.ipynb     ← original EDA
│   ├── 02_feature_engineering.ipynb
│   ├── 03_model_training.ipynb  ← original DNA-source classifier
│   └── 04 / 05_*.ipynb          ← placeholders from the older project iteration
├── .env / .env.example, .gitignore
├── requirements.txt
├── DOCS.md  (this file)
├── README.md (placeholder)
└── run code.txt   ← quick command cheatsheet
```

---

4. Data pipeline

4.1 Source data

`data/raw/disease_sequences/disease_dna_dataset.csv`

column	type	meaning
sequence	str	DNA sequence (variable length)
disease	str	Cancer / Diabetes / Alzheimers / Normal
id	str	NCBI accession or local id

Original counts: Cancer 377, Diabetes 164, Alzheimers 154, Normal 113 (total 808).

4.2 Audit (`src/data/audit.py`)

Run `python -m src.data.audit` to inspect:

- duplicate ratios per class
- distinct prefixes (catches "many shifted variants of the same gene")
- length statistics per class
- imbalance + length-leakage warnings

4.3 Deduplication (`src/data/dedup.py`)

- Drops exact duplicates.
- Computes MinHash-style signatures (top-64 hashed 8-mers) per sequence.
- Within each class, merges sequences whose Jaccard similarity ≥ 0.95 (keeps the longest).

After dedup we are left with ~500 unique parent sequences.

4.4 Augmentation (`src/data/augment.py`)

For every sequence:

- emit overlapping sliding-window fragments at 300, 600, 1200 bp with stride 50 %;
- compute reverse-complements;
- drop fragments shorter than 150 bp;
- de-duplicate fragments by hash to avoid identical chunks.

This dramatically increases positional invariance and supports short query sequences.

4.5 Build dataset (`src/data/build_dataset.py`)

End-to-end pipeline:

1. clean & validate sequences (only ACGT)
2. dedup (group-aware)
3. **group-aware split** — same parent sequence never crosses train/val (no leakage)
4. augment training set with full configuration; augment validation lightly
5. balance training set to the median class count (default ≈ 5712 per class → 22,848 total)
6. write `processed/train.parquet`, `processed/val.parquet`, `processed/build_summary.json`

Run: `python -m src.data.build_dataset`

---

5. ML models in detail

5.1 Sequence representation

DNA sequences are tokenized into **overlapping 6-mers** (k=6). For example `ATGCATG` → `atgcat`, `tgcatg`. Each sequence becomes a space-separated string of k-mers; that string is then vectorized with a **TF-IDF** transformer using 4-grams of k-mers (`ngram_range=(4, 4)`). One feature thus represents 4 consecutive 6-mers, i.e. a 24-base contextual window.

Why: full subsequences would cause vocabulary explosion. Single k-mers ignore context. 4-grams of k-mers strike a balance.

5.2 Classical ensemble (`src/ml/classifier.py`, `src/train.py`)

Soft-voting ensemble:

Estimator	Why
RandomForestClassifier (500 trees, balanced class weights)	robust to noise, gives feature_importances_ used for saliency
CalibratedClassifierCV(LinearSVC)	strong linear separator with calibrated probabilities
LogisticRegression (balanced)	simple linear baseline, low variance

Voting weights: `[2, 2, 1]`. Vectorizer settings: `min_df=3`, `max_df=0.98`, `sublinear_tf=True`, `norm='l2'`.

Training is fast (~34 seconds on CPU). **5-fold group-aware CV macro F1 = 0.96 ± 0.02.**

5.3 DNABERT-2 frozen + LR head (`src/ml/dnabert.py`)

`zhihan1996/DNABERT-2-117M` is a 117 M-parameter MosaicBERT genomic transformer with BPE tokenization. We use it as a **feature extractor**:

1. tokenize each sequence; for sequences > 510 tokens, use overlapping windows.
2. forward pass produces 768-dim hidden states; mean-pool over sequence length → one 768-vector per sequence.
3. cache embeddings on disk (`models/dnabert2_embeddings.npz`).
4. fit a **calibrated logistic-regression head** on top.

Frozen embeddings alone → macro F1 ≈ 0.66 — useful baseline but pretrained features are not disease-specific.

5.4 DNABERT-2 fine-tuned (`src/ml/dnabert_finetune.py`)

Custom PyTorch training loop tuned for 4 GB VRAM:

- `BertForSequenceClassification` head added directly on the transformer
- batch size 2, gradient accumulation 8 (effective 16)
- fp16 mixed precision via `torch.cuda.amp`
- AdamW + linear warmup
- max sequence length 256 tokens
- per-epoch evaluation with optimizer state moved to CPU between train and eval to avoid OOM
- best checkpoint by macro F1

Result: macro F1 ≈ 0.74 in 3 epochs (~5.8 min on RTX 3050). Long sequences are predicted via overlapping-window logit averaging.

5.5 Saliency (`src/ml/explain.py`)

For each base position we sum, over all overlapping vocabulary n-grams that include that base, the Random Forest's `feature_importances_` for the predicted class. Scores are then normalized to [0, 1].

The result is a per-base contribution score that the UI renders as colored DNA bases (slate → amber → rose).

5.6 OOD detection (`src/ml/ood.py`)

Cosine similarity between the query's TF-IDF vector and every training-set vector → take the **maximum** (nearest similarity). Compare against thresholds derived from the training corpus self-similarity distribution:

- `nearest >= P25` → low risk (in-distribution)
- `P5 <= nearest < P25` → medium risk (somewhat novel)
- `nearest < P5` → high risk (likely OOD; predictions unreliable)

5.7 Similarity search (`src/ml/similarity.py`)

Same TF-IDF vectors, now used to return the top-K nearest training sequences with their disease labels. This is a "BLAST-lite" sanity check: if your prediction says Cancer and the 5 closest known sequences are all Cancer, you can trust it.

5.8 Mutation analyser (`src/ml/mutation.py`)

Position-by-position diff between two equally-aligned sequences. For every mismatch:

- compute codon index = position // 3
- look up reference and variant codon (3 bases)
- translate to amino acid via `Bio.Seq.Seq.translate`
- categorize: synonymous / missense / nonsense / stop-loss / unknown
- run the classifier on both sequences and report **probability shifts** between reference and variant predictions.

---

6. API endpoints

All endpoints are JSON. Open `http://127.0.0.1:8000/docs` for the interactive Swagger UI.

Method	Path	Purpose
GET	`/`	Serves the SPA
GET	`/health`	Model status + classes
GET	`/diseases`	All disease info cards
GET	`/diseases/{name}`	Single disease detail
GET	`/models`	Available prediction backbones
POST	`/validate`	Sequence validation + composition stats
POST	`/stats`	Stats + ORFs
POST	`/predict`	Main prediction (with optional explain / OOD / similar)

Method	Path	Purpose
POST	`/mutation`	Compare reference vs variant
POST	`/batch`	FASTA file → many predictions
POST	`/report/pdf`	Downloadable PDF report
GET	`/chat/status`	RAG provider availability + KB stats
POST	`/chat`	RAG chat turn

The `model` field on `/predict` selects the backbone: `tfidf` (default) · `dnabert2` · `ensemble`.

---

7. Frontend pages

The SPA is mounted at `/`. All pages share a sidebar:

Page	Route	Purpose
Analyze	`analyze`	DNA prediction with saliency, OOD, similarity, ORFs, circular plot
Copilot	`copilot`	RAG chat with provider switch and context-aware prompts
Mutation Lab	`mutation`	Reference vs variant comparison
Batch Upload	`batch`	FASTA file → table of predictions
Knowledge Base	`knowledge`	Disease cards from `core/knowledge.py`
User Manual	`manual`	Step-by-step usage guide (this in-app help)
Presentation	`slides`	12-slide deck with arrow-key navigation
About	`about`	Project info + author credit

---

8. Configuration & secrets

Environment variables are loaded from `.env` (gitignored). See `.env.example`:

Variable	Required for	Where to get it
`GEMINI_API_KEY`	Copilot Gemini mode	https://aistudio.google.com/apikey (free)
`OLLAMA_BASE_URL`	Copilot Ollama mode	optional override; defaults to `http://localhost:11434`

---

9. How to run everything

First-time setup

```
# Create / activate venv, install deps
venv\Scripts\activate
pip install -r requirements.txt
```

```
# Install GPU PyTorch (replace cu124 with your CUDA version)
pip install torch --index-url https://download.pytorch.org/whl/cu124
```

Start the web app

```
venv\Scripts\activate
uvicorn src.api.app:app --reload --port 8000
# → open http://127.0.0.1:8000
```

Get the Gemini Copilot working

1. Get a free API key from https://aistudio.google.com/apikey
2. Edit `.env` and put `GEMINI_API_KEY=your_key_here`
3. Restart the server. The Copilot dropdown will show `GEMINI` as available.

Get the Ollama Copilot working

1. Install Ollama from https://ollama.com
2. `ollama pull llama3.2:3b`
3. `ollama serve`
4. Reload the page. The Copilot dropdown will show `OLLAMA` as available.

Run the CLI

```
python -m src.predict --sequence ATGGAT...
python -m src.predict --file my_sequences.fasta
```

---

10. How to retrain / extend

Re-build dataset & retrain TF-IDF ensemble

```
python -m src.data.audit                    # diagnose
python -m src.data.build_dataset            # build augmented splits
python -m src.train                         # retrain
```

Cross-validation + grid search

```
python -m src.eval cv --n-splits 5
python -m src.eval grid --ks 5 6 7
```

Results are written to `data/processed/cv_report_k*.json` and `grid_search_report.json`.

DNABERT-2 frozen pipeline

```
python -m src.ml.dnabert embed --source train     # encode dataset (~7 min on GPU)
python -m src.ml.dnabert train                    # train logistic head
```

DNABERT-2 fine-tuning

```
python -m src.ml.dnabert_finetune --epochs 3 --batch-size 2 --grad-accum 8
```

The fine-tuned model is saved to `models/dnabert2_finetuned/` and is automatically used by the `dnabert2` backbone whenever it exists.

Add new knowledge to the Copilot

Edit `src/rag/knowledge_base.py` and add new `Fact(...)` entries in `DISEASE_FACTS`, `GENE_FACTS`, or `METHOD_FACTS`. Then delete `models/rag_index.faiss` so the index rebuilds on next server start.

---

11. Evaluation results

TF-IDF ensemble — main classifier

- **5-fold group-aware CV macro F1:** 0.96 ± 0.02
- **Held-out test split macro F1:** 0.94
- **Per-class F1:** Cancer 0.94, Diabetes 0.90, Alzheimer's 0.92, Normal 1.00
- **Held-out hand-curated sequences (BRCA1, INS, APP, ACTB):** 4 / 4 correct

DNABERT-2 frozen + LR head

- **Macro F1:** 0.66
- **Held-out hand-curated:** 2 / 4

DNABERT-2 fine-tuned (3 epochs, balanced subsample)

- **Macro F1:** 0.74
- **Loss curve:** 1.25 → 0.79 → 0.48
- **Held-out hand-curated:** 3 / 4

Ensemble (TF-IDF + DNABERT-2 fine-tuned)

- **Held-out hand-curated:** 4 / 4

The ensemble is the most robust configuration on this dataset, while the TF-IDF backbone alone remains the most accurate single model thanks to the small dataset size.

---

12. Glossary

Term	Meaning
k-mer	Substring of length k drawn from the sequence with stride 1
n-gram of k-mers	n consecutive k-mers concatenated into one TF-IDF token
TF-IDF	Term-Frequency × Inverse-Document-Frequency feature vector
RF	Random Forest
SVM	Support Vector Machine (here `LinearSVC` with calibration)
LR	Logistic Regression
OOD	Out-of-distribution; sample is unlike anything in training
ORF	Open Reading Frame; an ATG-to-stop run that could code for a protein
Saliency	Per-base score measuring contribution to the prediction
RAG	Retrieval-Augmented Generation; LLM grounded by retrieved facts
FAISS	Facebook AI Similarity Search; fast vector index
FAISS / cosine	Inner product on L2-normalized vectors == cosine similarity
BPE	Byte Pair Encoding tokenization (used by DNABERT-2)
DNABERT-2	117 M-parameter genomic transformer pretrained on multi-species genomes
Group-aware split	Train/val split where every parent sequence belongs to exactly one side
MinHash Jaccard	Similarity estimate between two sets via hashed k-mer sketches
FP16	Half-precision floating point; halves GPU memory at small accuracy cost

---

Last updated automatically as part of the GenomeIQ Step 7 documentation pass.